

Julian Felipe Cardozo Costo

Documentación técnica del flujo n8n: Automatización de Consulta Judicial

1. Propósito

Este flujo automatiza la consulta judicial solicitada en la prueba técnica de TCC. La solución parte de un correo entrante, extrae el nombre de la persona a consultar, navega a la Rama Judicial mediante un navegador real, selecciona el sexto resultado público, descarga el documento del proceso y lo envía por correo con el archivo adjunto.

La implementación está construida en n8n y usa Playwright ejecutado desde un nodo ``Execute Command`` para resolver la interacción web dinámica con la página de la Rama Judicial, que requiere JavaScript y elementos renderizados en navegador.

2. Herramienta utilizada y justificación

La herramienta principal es n8n, en modalidad self-hosted, con integración de Gmail, nodos Code, HTTP Request, Read/Write Files y Execute Command.

La decisión de usar Playwright dentro del flujo responde a que la consulta de la Rama Judicial no es confiable con scraping estático ni con automatización por coordenadas. El workflow declara explícitamente que esta decisión evita depender de HTML incompleto y mejora la estabilidad frente a una interfaz dinámica.

3. Arquitectura general

La arquitectura implementada en el flujo sigue esta secuencia lógica:

1. Gmail detecta un correo entrante filtrado por bandeja y asunto.
2. Se obtiene el mensaje completo desde Gmail para acceder al cuerpo real del correo.
3. Un nodo Code prepara la entrada, extrae el nombre y valida parámetros de ejecución.
4. Un nodo ``Execute Command`` ejecuta Playwright para navegar en la Rama Judicial y descargar el documento.

5. Otro nodo Code valida la salida del RPA y normaliza los datos de negocio.
6. El archivo descargado se lee como binario para reutilizarlo dentro del flujo.
7. El documento se sube a Supabase Storage como respaldo opcional.
8. Se genera una URL temporal del archivo en Supabase.
9. Se prepara el correo final con asunto, cuerpo, adjunto y datos de trazabilidad.
10. Se envía el correo de salida mediante Gmail.

4. Nodos del workflow

4.1 Disparadores

El flujo tiene dos entradas. La ruta productiva comienza en `Gmail Trigger - Producción (configurar y activar)`, configurado para revisar `INBOX`, filtrar mensajes no leídos y aplicar la consulta `subject:"Prueba RPA Judicial" is:unread`.

También existe una ruta de prueba manual compuesta por `Ejecutar prueba manual` y `Simular correo Gmail`. Esta rama está deshabilitada y sirve para pruebas controladas sin depender de una bandeja real.

4.2 Lectura del correo

El nodo `Gmail - Obtener correo completo` usa la operación `get` sobre el `messageId` recibido desde el trigger para recuperar el contenido completo del mensaje.

Este paso es necesario porque la extracción del nombre no se hace desde un resumen del correo, sino desde el payload completo entregado por Gmail.

4.3 Preparación de entrada y validaciones

El nodo `Preparar entrada, validar nombre y parámetros` concentra varias responsabilidades críticas. Define variables de configuración, normaliza el cuerpo del

correo, decodifica contenido Base64URL cuando aplica, limpia HTML y espacios, intenta detectar un nombre válido y prepara metadatos como fecha de ejecución, nombre de archivo esperado, asunto del correo y ruta de almacenamiento.

Este nodo también valida que existan parámetros esenciales como ``recipientEmail``, ``baseUrl``, ``chromiumPath``, ``supabaseUrl`` y ``supabaseServiceRoleKey``. Si falta una configuración crítica, el flujo falla temprano con un mensaje explícito.

La detección del nombre utiliza una estrategia mixta: primero intenta encontrar patrones explícitos dentro del cuerpo del correo y, como respaldo, usa una coincidencia general de texto con varias palabras. Además, exige que el nombre contenga al menos dos componentes antes de continuar.

4.4 Ejecución del RPA web

El nodo ``Ejecutar RPA Playwright - Shell`` ejecuta un script Node.js con Playwright desde ``Execute Command``. El script abre Chromium, navega a ``https://consultaprocesos.ramajudicial.gov.co/Procesos/NombreRazonSocial``, selecciona la opción de consulta completa, configura ``Tipo de Persona`` en ``Natural``, ingresa el nombre consultado y dispara la búsqueda.

La selección del sexto resultado no está hardcodeada por coordenadas. El script inspecciona filas visibles, descarta encabezados o filas no válidas, detecta radicados públicos, construye una lista de resultados utilizables y toma el sexto elemento real de esa colección.

Una vez abierto el detalle del proceso, el script busca la acción ``Descargar DOC``, espera el evento real de descarga del navegador y guarda el archivo localmente en la ruta configurada. Después valida que el archivo exista, tenga tamaño mínimo y no corresponda al listado general sino al detalle del proceso esperado.

Durante la ejecución se generan logs en formato JSONL y capturas de pantalla en etapas relevantes, incluyendo página inicial, formulario diligenciado, resultados, detalle del proceso y captura final de éxito o error.

4.5 Validación posterior del RPA

El nodo `Validar RPA y preparar Supabase` toma la salida estándar del comando y extrae el JSON delimitado entre `\_\_RPA\_RESULT\_START\_\_` y `\_\_RPA\_RESULT\_END\_\_`. Si no encuentra ese bloque o si el JSON no es válido, genera un error de ejecución con evidencia del `stdout` y `stderr`.

Si el RPA reporta éxito, el nodo construye variables reutilizables para el resto del flujo, incluyendo asunto y cuerpo de correo con nombre consultado, fecha, URL del proceso y archivo generado.

4.6 Manejo del archivo descargado

El nodo `Leer DOC descargado` toma la ruta local del documento generado y lo convierte a binario dentro de n8n, usando `application/msword` como tipo MIME.

Esa conversión permite adjuntar el archivo al correo y reutilizarlo en la carga a almacenamiento externo sin depender nuevamente del sistema de archivos.

4.7 Respaldo en Supabase

El flujo incluye dos nodos HTTP para persistencia opcional. `Subir DOC a Supabase Storage` envía el binario al bucket configurado mediante la Storage API con `x-upsert=true`, y `Crear URL temporal Supabase` solicita una signed URL con vigencia parametrizable.

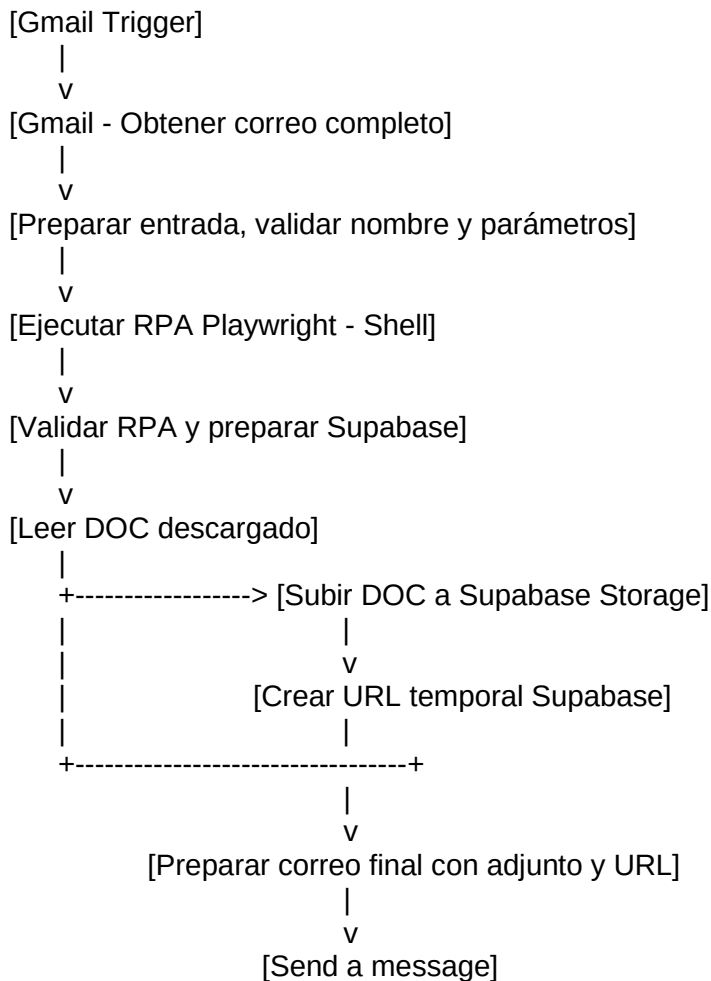
Esta parte no es obligatoria para cumplir la prueba, pero agrega trazabilidad y respaldo documental. El propio diseño del flujo deja claro que el requisito mínimo sigue siendo el envío del correo con el documento adjunto.

4.8 Preparación y envío del correo final

El nodo `Preparar correo final con adjunto y URL` consolida datos del archivo leído, la validación del RPA y la respuesta de Supabase para construir un texto final con nombre consultado, fecha, URL del proceso, radicación, ubicación del archivo y enlace temporal si fue generado.

El nodo `Send a message` usa Gmail para enviar el correo final con el asunto parametrizado, cuerpo en HTML y el documento descargado como adjunto binario.

5. Flujo paso a paso



La secuencia anterior corresponde a la ruta principal observada en las conexiones del workflow. El flujo manual reutiliza el mismo núcleo funcional, pero reemplaza el trigger productivo por un correo simulado.

6. Configuración requerida

La configuración principal está centralizada en el nodo `Preparar entrada, validar nombre y parámetros`. Allí se observan campos para correo destino, correo remitente, URL base de consulta, carpetas locales de descargas, logs y screenshots, modo headless, timeout, ruta del ejecutable Chromium, URL de Supabase, bucket, service role key y tiempo de expiración de la signed URL.

Las credenciales requeridas son Gmail OAuth2 para lectura y envío de correo, además de una cuenta SMTP o Gmail según la estrategia de entrega que se documente en el repositorio. El propio flujo recomienda activar el `Gmail Trigger` solo después de validar correctamente la ejecución manual.

7. Manejo de errores y logging

La solución implementa manejo básico de errores en varios niveles. Antes de navegar, valida nombre y parámetros críticos; durante la ejecución web, registra eventos en log estructurado; al final, verifica que el RPA haya devuelto JSON válido y un estado exitoso.

El archivo de log se escribe como `ejecuciones\_rpa\_consulta\_judicial.jsonl`, lo que facilita auditoría por ejecución. Además, el flujo captura screenshots en éxito y error, lo que fortalece la defensa técnica durante la demo.

8. Decisiones técnicas clave

Extracción del nombre

La extracción del nombre se realiza a partir del cuerpo real del correo, no del asunto ni de un campo manual. El flujo decodifica partes del payload de Gmail, limpia HTML, normaliza espacios y valida que el texto extraído represente un nombre utilizable.

Identificación del sexto resultado

La identificación del sexto resultado se resuelve dinámicamente mediante análisis de filas visibles y radicados detectados en pantalla. Esta decisión evita depender del primer resultado, de posiciones absolutas o de selectores demasiado frágiles.

Navegación estable

La navegación usa Playwright con esperas basadas en carga del DOM, `networkidle`, visibilidad de elementos y validación del detalle del proceso. Esta aproximación es más robusta que usar tiempos fijos de espera.

Descarga y validación del documento

La descarga no se da por exitosa solo por hacer clic. El script espera el evento de descarga, guarda el archivo, revisa tamaño mínimo y valida que el contenido corresponda al detalle consultado.

9. Cumplimiento frente a la prueba

El flujo satisface el núcleo del reto: detecta correo, extrae nombre, consulta la Rama Judicial, selecciona el sexto resultado, descarga el documento y envía un correo con adjunto.

Adicionalmente, incorpora una mejora no obligatoria mediante Supabase Storage y signed URL temporal. Esa parte debe presentarse como valor agregado y no como requisito central de cumplimiento.

10. Riesgos, limitaciones y mejoras

La principal limitación es la dependencia de cambios en la interfaz de la Rama Judicial. Si cambian textos, estructura visual, selectores o comportamiento de descarga, será necesario ajustar el script Playwright.

También existe una inconsistencia documental potencial entre `.doc` y `.docx`, porque el flujo habla de `Descargar DOC` pero parte del script manipula nombres de archivo en formato `.docx`. Ese punto debe corregirse o explicarse antes de la entrega final.

Como mejoras futuras, conviene mover configuraciones sensibles a variables externas o credenciales seguras, eliminar secretos hardcodeados del JSON, incorporar reintentos automáticos y permitir procesamiento de múltiples correos por ejecución.

11. Estructura de entrega recomendada

```
/rpa-consulta-judicial
/flujo
  RPA-Consulta-Judicial-Gmail-Supabase-Adjunto-Version-Estandar-3.json
/screenshots
  success_*.png
  01_pagina_inicial_*.png
  02_formulario_diligenciado_*.png
  03_resultados_*.png
  04_detalle_proceso_*.png
/logs
  ejecuciones_rpa_consulta_judicial.jsonl
README.md
```

La estructura anterior coincide con las notas internas del propio workflow y con los entregables solicitados en la prueba técnica.